



deti

universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

Curso 8316 - Licenciatura em Engenharia de Computadores e Informática
Disciplina 43835 - Sensores e Sinais
Ano letivo 2025/2026

Relatório

Mini-Projecto: Sistema de Controlo de Acesso via Tons

Autores:

120152 Tiago Pita

126323 Martin Oliveira Pereirinha

Turma P1 Grupo [Grupo]

Data 31/05/2026

Docente Pedro Fonseca

Resumo: O seguinte documento descreve o desenvolvimento de uma fechadura eletrónica baseada no reconhecimento acústico de sons. Este sistema caracteriza-se pela presença de uma sequência de 4 algarismos, cujos valores variam entre 0 e 2. A captação do áudio, através de um microfone analógico SPW2430, utiliza três filtros digitais FIR passa-banda de ordem $N = 100$, de maneira a isolar as frequências alvo dos três valores (0, 1 e 2) e ocorre de forma contínua, a uma frequência de 20kHz. Posteriormente, com a ajuda de um microcontrolador ESP32-C6, foi criada uma máquina de estados finita (FSM), que com os valores obtidos, faz a deteção de cada tom e a análise da sequência a que são apresentados, sendo capaz de detetar a introdução de sequências erradas ou corretas. Além das mensagens no terminal do sistema, foi utilizado um LED, para melhor visualização dos resultados, fornecendo um feedback visual ao utilizador.

Lista de Siglas e Abreviações

ADC	Analog-to-Digital Converter
Amp	Amplificar
CPU	Central Processing Unit
DC	Corrente Contínua
DTMF	Dual-Tone Multi-Frequency
ESP32-C6	Espressif System Platform 32 - C6
FIR	Finite Impulse Response
Freq.	Frequência
FSM	Finite State Machine
GPIO	General Purpose Input/Output
IDF	IoT Development Framework
IoT	Internet of Things
LED	Light Emitting Diode
Mic	Microfone
NMEC	Número Mecanográfico
RTOS	Real-time Operating System

Introdução

O seguinte trabalho tem como objetivo a captação de um sinal sonoro, através de um microfone, para simular o funcionamento de uma fechadura eletrônica, codificada com uma sequência de números. Por questões de simplicidade, a sequência, consiste em 4 algarismos, entre 0 e 2, inclusive, com a presença de algarismos repetidos. Cada um destes algarismos está codificado numa determinada frequência, que é única para cada algarismo, permitindo a distinção do mesmo através do sinal obtido.

Para a captação do sinal sonoro, além do microfone, foi utilizada a ferramenta Octave, para desenvolvimento de três filtros digitais, capazes de filtrar a informação útil, nomeadamente as frequências de cada um dos três números utilizados: 0, 1 e 2. Posteriormente, para analisar o sinal obtido, implementar o funcionamento dos filtros e finalmente desenvolver o mecanismo de simulação da fechadura, foi utilizado um microcontrolador Espressif 32 C6 (ESP32-C6).

Por fim, este microcontrolador foi ligado através por fios, a um Light Emitting Diode (LED), que permitia fornecer um feedback visual. Quando a porta estava fechada, este LED encontrava-se apagado, já quando a porta se encontrava aberta, este ligava durante um curto período de tempo e por fim, quando se introduzia uma sequência errada, este piscava durante 5 segundos.

A Figura 1. abaixo contém os componentes e as ligações utilizadas para o projeto.

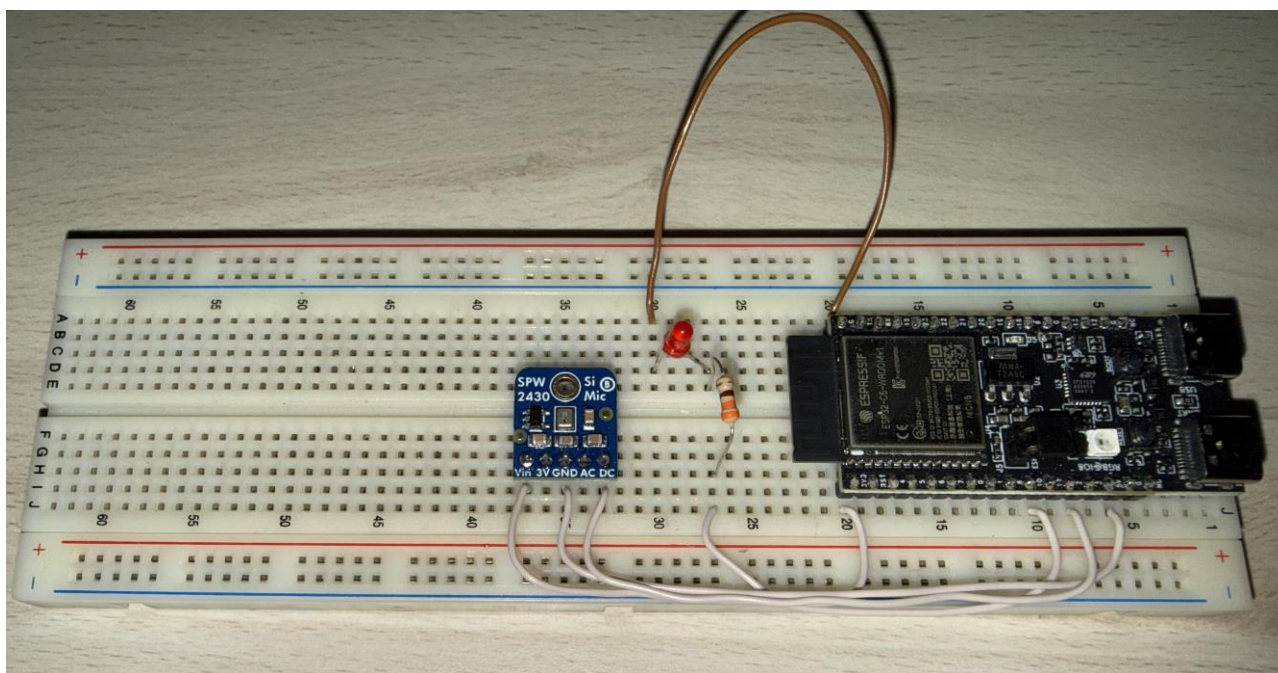


Figura 1. Esquema de montagem do mecanismo. A figura mostra uma *breadboard* com o microcontrolador ESP32-C6, o Microfone SPW2430, uma resistência e um LED, juntamente com as ligações entre os vários componentes.

Descrição do problema e objetivos

O objetivo principal deste trabalho consiste na captação e análise de um sinal sonoro para emular o funcionamento de uma fechadura eletrônica controlada por uma palavra-passe de quatro dígitos, aplicando na prática os conceitos de sinalização *in-band* (à semelhança dos sistemas DTMF). Para a concretização deste projeto, foram definidos os seguintes objetivos específicos:

ID	Nome do Objetivo	Definição do Objetivo
O1	Captação do Sinal	Adquirir um sinal sonoro de forma contínua através de um microfone analógico e processar os blocos de amostras no microcontrolador em tempo-real.
O2	Amostragem do Sinal	Converter o sinal contínuo num sinal discreto, garantindo que a frequência de amostragem respeita com segurança o Teorema de Nyquist.
O3	Filtragem do Sinal	Dimensionar e implementar filtros digitais FIR passa-banda para processar o sinal discreto, isolando e identificando as frequências únicas dos três símbolos (0, 1 e 2) da sequência.
O4	Simulação da fechadura	Implementar a lógica de controlo (Máquina de Estados) capaz de validar a sequência introduzida e fornecer feedback visual ao utilizador através de um LED e mensagens via consola.
O5	Análise e tratamento de dados	Avaliar o comportamento do sistema perante a introdução de sequências corretas e incorretas, testando a imunidade dos filtros ao ruído de fundo e a estabilidade da temporização.

Tabela 1. Objetivos definidos para realizar a implementação do sistema de controlo de acesso.

Projeto e conceção

Cálculo das Frequências Alvo

De acordo com os requisitos estabelecidos no enunciado do projeto, o sistema desenvolvido deve ser capaz de decodificar três tons distintos de frequência única, mapeados respetivamente para os símbolos acústicos "0", "1" e "2". Estas frequências de operação são calculadas de forma personalizada com base no enquadramento académico do grupo (Turma Prática P1) e nos respetivos Números Mecanográficos (NMECs: 120152 e 126323). As frequências obtidas para cada símbolo encontram-se na Tabela 2. abaixo.

Símbolo	Cálculo da Frequência	Frequência obtida
0	$1 \times 500 = 500 \text{ Hz}$	500 Hz

1	$(1 + 2 + 0 + 1 + 5 + 2) + (1 + 2 + 6 + 3 + 2 + 3) =$ $11 + 17 = 28 = 8_{10}$ $500 + 700 + (8 \times 20) = 1360 \text{ Hz}$	1360 Hz
2	$1360 + 700 + (8 \times 20) = 2220 \text{ Hz}$	2220 Hz

Tabela 2. Determinação das frequências para cada um dos símbolos.

Uma vez que as frequências alvo estão bastante separadas umas das outras (com mais de 800 Hz de diferença entre si). Isto é uma vantagem para o sistema, porque significa que os filtros não precisam de ter um corte extremamente agressivo para evitar que o sinal de um tom seja confundido com o outro.

Dimensionamento dos Filtros FIR

Para isolar cada um destes tons, cumprindo as especificações exigidas para o projeto, foram desenvolvidos e implementados três filtros digitais FIR (Finite Impulse Response) do tipo Passa-Banda. Por outras palavras, filtros que apenas permitem a passagem de uma das gamas do sinal apresentado. A obtenção dos coeficientes, foi utilizada com recurso à técnica de janelamento, minimizando eventuais descontinuidade nas bordas. Este processo foi realizado através da função *fir1()* da ferramenta Octave.

A frequência de amostragem (f_s) do sistema foi fixada em 20 kHz através da configuração da ADC do microcontrolador ESP32-C6 no modo contínuo, sendo a frequência de Nyquist ($f_{Nyquist} = f_s/2$) considerada de 10kHz, evitando o fenómeno de *aliasing* e garantindo a obtenção de amostras com valores corretos.

De acordo com o funcionamento da função *fir1()*, as frequências de corte devem ser fornecidas na forma de um valor normalizado (W_n) entre 0 e 1, onde 1 corresponde à frequência de Nyquist, utilizando-se assim a seguinte equação:

$$W_n = \frac{f}{f_{Nyquist}} = \frac{f}{f_s/2}$$

Definiu-se ainda, para cada filtro, uma largura de banda de 200 Hz fornecendo uma margem de ± 100 Hz à volta da frequência central de cada tom. Posteriormente, aplicando a equação, foram obtidos os seguintes intervalos normalizados para gerar os três filtros digitais, que se encontram na tabela 3 abaixo.

Símbolo	Frequências de Corte		Cálculo do W_n
	Freq. 1	Freq. 2	
0	400	600	$W_{n0} = \left[\frac{400}{10000}, \frac{600}{10000} \right] = [0.04, 0.06]$
1	1260	1460	$W_{n1} = \left[\frac{1260}{10000}, \frac{1460}{10000} \right] = [0.126, 0.146]$
2	2120	2320	$W_{n2} = \left[\frac{2120}{10000}, \frac{2320}{10000} \right] = [0.212, 0.232]$

Foi ainda necessário efetuar a escolha da ordem (N) do filtro. A escolha de uma ordem

muito baixa, resultaria no filtro a deixar passar demasiado ruído, já na escolha de uma ordem demasiado alta, o ESP32 não teria capacidade de processamento para executar as multiplicações todas em tempo real, ultrapassando o limite arquitetural de 200 coeficientes definido no código (*MAX_FILT_IR_LEN*). Dessa forma, foi escolhida como ordem dos filtros $N = 100$ garantindo 101 coeficientes por filtro, cortando o ruído indesejado e permitindo que o ESP32 operasse dentro de um tempo aceitável.

Implementação e Processamento Contínuo

A aquisição do sinal de áudio pelo microcontrolador é feita em blocos de 2048 amostras. No entanto, a operação de convolução de um filtro FIR com ordem $N = 100$ exige a memória do sinal. Isto significa que, para calcular a primeira amostra de um novo bloco, o filtro necessita das 100 amostras imediatamente anteriores, de modo a evitar distorções ou frequências fantasma nas fronteiras de cada bloco. Em cada ciclo do *FreeRTOS*, foram guardadas as últimas 100 amostras do bloco atual, ocorrendo uma concatenação dessas 100 amostras no ciclo seguinte com as 2048 novas amostras num buffer de trabalho temporário com um comprimento total de 2148 amostras. Isto com o objetivo de garantir a continuidade do sinal no domínio do tempo antes da aplicação da função *dsps_conv_f32()*, sendo o resultado da convolução um sinal digital filtrado.

Para que a máquina de estados consiga tomar uma decisão lógica, transformamos esse *array* de amostras num valor escalar que representa a **potência média** daquela banda de frequências, eliminando a dependência do sinal oscilar entre valores positivos e negativos. Para o fazer, aplicámos a equação da potência média presente no formulário da unidade curricular:

$$P = \frac{1}{L} \sum_{n=100}^{2148-1} (y[n])^2$$

onde $y[n]$ representa o sinal de saída da convolução e $L = 2048$ é o número de amostras úteis analisadas (excluindo os primeiros 100 pontos correspondentes à região de transição da convolução). Este valor de potência é calculado para as saídas dos três filtros, sendo de seguida, o valor obtido comparado com um limiar (*threshold*) de decisão determinado através de testes práticos com a placa. Se a potência calculada para o filtro do símbolo "1", por exemplo, ultrapassar esse limiar, o sistema regista a deteção desse tom específico.

Máquina de Estados e Validação da Sequência

Dado que cada buffer de áudio corresponde a aproximadamente 100 ms de som, um único tom reproduzido durante um segundo iria atravessar vários ciclos de processamento, gerando múltiplas deteções consecutivas do mesmo símbolo. Para interpretar os comandos do utilizador e contornar este problema, a solução passou por implementar uma Máquina de Estados Finitos (FSM) que garante a deteção isolada de cada tom, forçando a existência de uma pausa (silêncio) entre os símbolos introduzidos. A FSM é composta por três estados estruturais: dois operacionais — **WAIT_SOUND** (à espera de um som dominante que supere o limiar) e **WAIT_SILENCE** (bloqueio de novas deteções até ao retorno do silêncio) — e um estado de segurança — **WAIT_PENALTY**, responsável por gerir o tempo de rejeição do sistema de forma síncrona com o escalonador do RTOS.

No estado inicial (*WAIT_SOUND*), o sistema avalia se alguma das potências calculadas ultrapassa o limiar de ruído (*threshold*). Quando ocorre uma deteção, é aplicada uma lógica de seletividade comparativa: avaliamos qual dos três filtros apresenta o maior valor de potência no momento. Esta abordagem garante que o sistema escolhe o tom dominante e rejeita eventuais ruídos de fundo (como sons de banda larga ou impactos) que possam ter atravessado as bandas

dos restantes filtros.

Identificado o símbolo válido (0, 1 ou 2), a máquina transita imediatamente para o estado *WAIT_SILENCE*. Neste estado, o sistema suspende novas detecções e apenas regressa ao estado inicial quando a potência dos três filtros cair abaixo do limiar, o que atesta que o utilizador terminou efetivamente a reprodução do tom.

Para guardar a sequência inserida, foi utilizado um registo de deslocamento (*shift register*) com quatro posições. Sempre que um tom válido é detetado, os valores guardados movem-se uma posição para a esquerda, descartando o tom mais antigo, e o novo é guardado na última posição.

Finalmente, o vetor atualizado é comparado com as chaves predefinidas do nosso sistema:

- Se a sequência corresponder ao comando de abertura (1, 2, 0, 0), o microcontrolador aciona o GPIO respetivo para ligar o LED (simulando a porta aberta).
- Se a sequência corresponder ao comando de fecho (2, 0, 1, 1), o LED é desligado.
- Se a sequência estiver preenchida com quatro dígitos, mas não corresponder a nenhuma das chaves, o sistema reconhece o erro e emite um alerta visual (LED a piscar).

Independentemente de a sequência avaliada estar correta ou errada, sempre que o buffer de quatro posições é totalmente preenchido e processado, o vetor de memória é limpo (preenchido com valores inválidos, como -1). Isto garante que o utilizador tem de introduzir uma sequência totalmente nova na tentativa seguinte.

Sinalização Visual da Penalidade

No cenário de uma palavra-passe inválida, o sistema aciona um estado de penalização, colocando o LED a piscar durante um período de 5 segundos, obrigando o utilizador a aguardar antes de uma nova tentativa.

Do ponto de vista arquitetural, implementar este atraso através de uma função de bloqueio síncrono (como o `vTaskDelay()`) revelar-se-ia um erro. O periférico de aquisição de áudio continua a injetar sem interrupção blocos de amostras na fila (Queue) a cada ~ 100 ms. Se a tarefa de processamento (`pv_processor_task`) fosse suspensa durante 5 segundos, dezenas de buffers acumular-se-iam na memória, provocando um overflow da fila, colapsando o sistema por esgotamento de RAM.

Para solucionar este desafio, expandimos a Máquina de Estados Finitos com um terceiro estado: *WAIT_PENALTY*. Quando uma sequência incorreta é validada, o sistema regista o tempo atual utilizando o relógio interno do RTOS (`xTaskGetTickCount()`) e transita para este estado de penalização.

Durante o estado *WAIT_PENALTY*, a tarefa de processamento continua a executar o seu ciclo normal: recebe os blocos de amostras da Queue (garantindo que a memória é libertada), mas a FSM ignora o processamento do sinal, desviando o fluxo lógico apenas para calcular o tempo decorrido. O piscar do LED é gerido de forma matemática, sem necessidade de interrupções de hardware, avaliando o diferencial de tempo a cada ciclo e alternando o estado do GPIO consoante um intervalo predefinido de sinalização.

Ultrapassado o limite temporal de 5 segundos estipulado, o sistema termina o alarme visual, efetua o reset da sequência e transita autonomamente para o estado *WAIT_SOUND*.

Condicionamento do Sinal Digital

Ao realizar testes com o sinal captado pelo microfone SPW2430 deparámo-nos com uma

amplitude reduzida e adicionalmente a uma forte componente contínua (*DC Offset*), resultante da sua tensão de alimentação. Para garantir a eficácia da filtragem, implementou-se uma fase de condicionamento prévio na tarefa de processamento.

Primeiramente, procede-se à eliminação do desvio DC, calculando a média das 2048 amostras do buffer e subtraindo esse valor individualmente a cada amostra, centrando a onda sonora perto zero:

$$x_{centrado}[n] = x[n] - \mu$$

De seguida, aplica-se uma amplificação por software através da multiplicação do sinal centrado por um fator constante ($SOFTWARE_AMP = 10$).

$$x_{centrado}[n] = (x[n] - \mu) * SOFTWARE_AMP$$

Otimização Computacional e Tempo-Real

Durante os testes, detetou-se um atraso acentuado na resposta do sistema a eventos externos. O cálculo das três convoluções FIR em blocos de 2148 amostras exigia um esforço de processamento intenso do microcontrolador, bem como o excesso de prints no terminal. Para recuperar a execução em tempo-real e otimizar a performance, implementou-se um desvio lógico no código: sempre que a Máquina de Estados transita para o estado de penalidade, os cálculos matemáticos dos filtros são ignorados. Esta otimização libertou a carga do CPU, permitindo ao sistema operativo gerir a temporização do LED de forma exata e evitar a acumulação excessiva de amostras na fila de receção de áudio.

Resultados

Reprodução	Símbolo 0 (500Hz)	Símbolo 1 (1360Hz)	Símbolo 2 (2220Hz)
Silêncio (Ruido de Fundo)	181.608688	122.601898	125.732452
Tom 500Hz	185.979919	123.610619	126.774818
Tom 1360Hz	182.951523	132.206985	126.612190
Tom 2220Hz	181.642029	122.628319	140.722412

Tabela 4. Potência média antes da aplicação do “Condicionamento do Sinal Digital”

Reprodução	Símbolo 0 (500Hz)	Símbolo 1 (1360Hz)	Símbolo 2 (2220Hz)
Silêncio (Ruido de Fundo)	3.891622	3.039868	3.206163
Tom 500Hz	91.098824	2.908597	3.006300

Tom 1360Hz	4.096170	95.744576	3.195140
Tom 2220Hz	3.168117	3.697569	198.802383

Tabela 5. Potência média depois da aplicação do “Condicionamento do Sinal Digital”

```

I (48881) MIC_EXAMPLE: Tom detetado: 0 | Sequencia atual: [1, 2, 0, 0]
I (48881) MIC_EXAMPLE: COMANDO ABRIR DETETADO!
I (68611) MIC_EXAMPLE: A entrar em WAIT_SILENCE. Aguarde...
...
I (68611) MIC_EXAMPLE: Tom detetado: 2 | Sequencia atual: [1, 2, 0, 2]
W (68611) MIC_EXAMPLE: SEQUENCIA ERRADA!
I (68611) MIC_EXAMPLE: A entrar em WAIT_SILENCE. Aguarde...
...
I (73731) MIC_EXAMPLE: Penalizacao terminada. Tentar novamente.
...
I (88011) MIC_EXAMPLE: Tom detetado: 1 | Sequencia atual: [2, 0, 1, 1]
I (88011) MIC_EXAMPLE: COMANDO FECHAR DETETADO!
I (68611) MIC_EXAMPLE: A entrar em WAIT_SILENCE. Aguarde...

```

Figura 2. Informações enviadas para o terminal com base em algumas das possíveis sequências recebidas pelo microfone, bem como o seu efeito no sistema.

Análise dos Resultados

Análise do Impacto do Condicionamento de Sinal e Equalização

A análise inicial dos dados brutos revelou um comportamento inesperado do sistema. Como documentado na primeira tabela de resultados, a reprodução de um tom de 500 Hz elevava a potência medida de 181.6 (ruído de fundo) para apenas 185.9. Esta diferença de potência praticamente nula causou estranheza durante a fase de testes, impossibilitando a criação de um limiar (*threshold*) de decisão fiável entre o silêncio e um comando sonoro válido.

Para tentar perceber este problema, pesquisámos sobre o comportamento do nosso hardware. Percebemos que este fenómeno se devia a dois fatores. Primeiro, o sinal de áudio captado pelo microfone em bruto era muito fraco. Segundo, existia uma tensão de alimentação contínua muito alta gerada pelo microfone (conhecida como *DC Offset*). Como este valor constante, que anda à volta dos 0 Hz, está muito próximo do nosso primeiro filtro de 500 Hz, parte dessa energia contínua acabava por interferir nos cálculos desse filtro, aumentando o valor do ruído de fundo de forma artificial.

A implementação do bloco de condicionamento digital resolveu de forma eficaz esta limitação. Ao forçar a centragem do sinal em zero e aplicar o ganho digital antes do cálculo da potência, a componente DC foi praticamente eliminada. Assim, como evidenciado na segunda tabela de resultados, a potência do ruído de fundo colapsou para valores quase nulos (na ordem de 3.0), enquanto a potência dos tons válidos se destacou, permitindo à Máquina de Estados operar com margens de decisão mais seguras.

Durante os ensaios finais de validação, identificou-se uma assimetria física no hardware de emissão. À mesma distância e com o mesmo volume, o altifalante do *smartphone* utilizado para testes reproduzia os tons agudos (2220 Hz) com significativamente mais eficiência do que os tons graves (500 Hz). Esta discrepância resultava em picos de potência desiguais.

De modo a uniformizar a detecção e evitar a implementação de múltiplos limiares de decisão na lógica de controlo, optou-se por aplicar uma equalização por software no processamento final. Utilizando a potência máxima observada experimentalmente no Símbolo 2 como referência base, as energias calculadas foram niveladas através da multiplicação dos seguintes números:

- Potência Símbolo 0 (500 Hz): Multiplicada por 10.0
- Potência Símbolo 1 (1360 Hz): Multiplicada por 5.0
- Potência Símbolo 2 (2220 Hz): Multiplicada por 1.0 (Referência)

Esta calibração garantiu que o sistema reage de uma forma mais homogênea a qualquer um dos símbolos do comando.

Validação Funcional do Sistema

A Máquina de Estados encontra-se totalmente funcional. A validação foi realizada com recurso ao altifalante de um *smartphone* no volume máximo, posicionado próximo do microfone. Através do áudio gerado, é possível acompanhar o fluxo correto das transições entre a detecção dos tons válidos e o silêncio obrigatório.

Conclusões

O projeto cumpriu com sucesso todos os requisitos propostos para o desenvolvimento de um sistema de controlo de acessos baseado no reconhecimento de padrões acústicos, implementado num microcontrolador ESP32-C6. A aquisição contínua do sinal e a sua filtragem no domínio do tempo provaram ser uma metodologia viável e robusta. A escolha de filtros digitais FIR Passa-Banda de ordem $N=100$ permitiu um isolamento eficaz das frequências-alvo (500 Hz, 1360 Hz e 2220 Hz), garantindo uma elevada seletividade face ao ruído de fundo do ambiente.

O desenvolvimento prático evidenciou a importância crítica do condicionamento do sinal digital para contornar as limitações físicas do hardware. A presença de um forte *DC Offset* gerado pelo microfone SPW2430, a remoção desta componente contínua, aliada à aplicação de um ganho digital e de pesos de equalização, homogeneizou a resposta do sistema perante a eficiência assimétrica do altifalante utilizado. Do ponto de vista arquitetural e de lógica de controlo, a implementação de uma Máquina de Estados Finitos assegurou a detecção de eventos únicos e validou corretamente as passwords introduzidas. O problema da latência foi solucionado através de uma otimização computacional: o sistema contorna o cálculo pesado das convoluções durante o estado de penalização, permitindo ao microcontrolador gerir a sinalização visual de erro sem causar estrangulamento no CPU.

Em suma, conceção matemática dos filtros e as estratégias definidas culminou num instrumento de detecção estável.

Referências e Bibliografia

Adafruit Silicon MEMS Microphone Breakout – SPW2430, Adafruit, Visualizado a 29 de maio de 2026, em <https://www.adafruit.com/product/2716>

DC offset, Audacity, Visualizado a 29 de maio de 2026, em https://manual.audacityteam.org/man/dc_offset.html

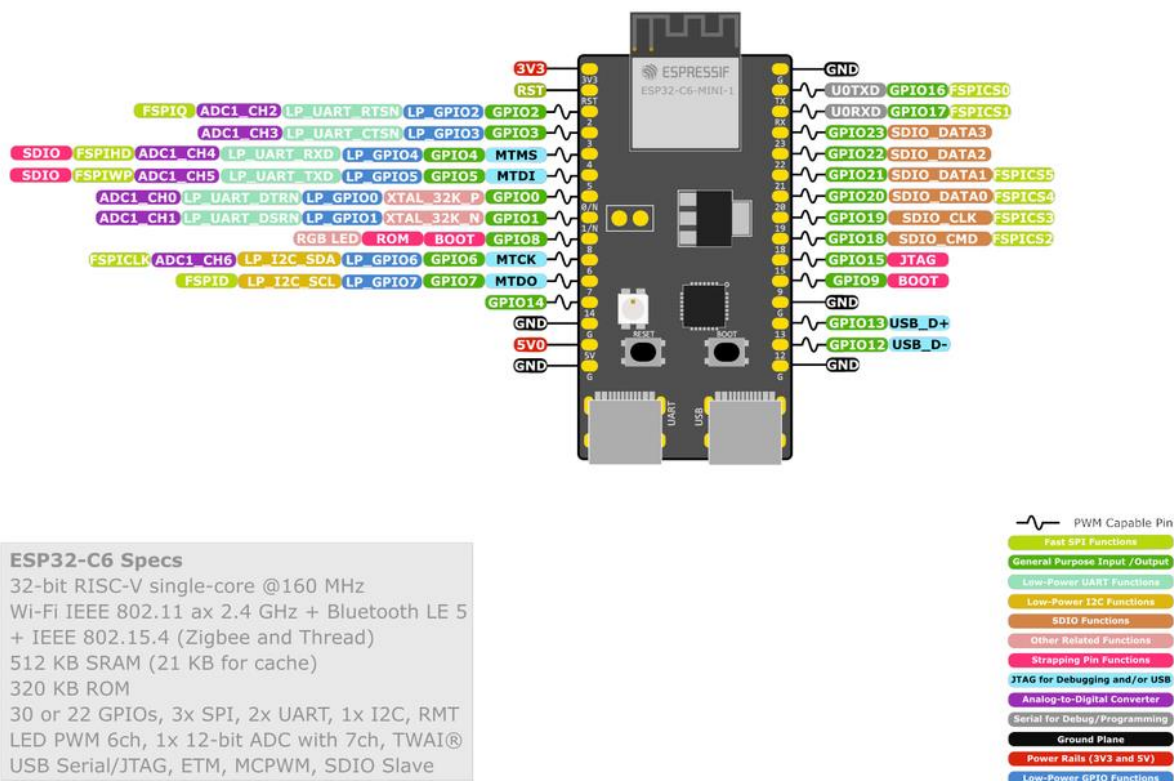
Espboards Dev, Espressif ESP32-C6-DevKitM-1 Pinout Diagram, Visualizado a 30 de maio de 2026, em <https://www.espboards.dev/esp32/esp32-c6-devkitm-1/>

Espressif, Component ADC Mic, Visualizado a 25 de maio de 2026, em https://components.espressif.com/components/espressif/adc_mic/versions/0.2.3/readme?language=en

Material fornecido na plataforma e-Learning e nas aulas da unidade curricular

Anexos

ESP32-C6-DevKitM-1



Anexo 1. Esquema pinout do microcontrolador Espressif ESP32-C6. Obtido de <https://www.espboards.dev/esp32/esp32-c6-devkitm-1/>